

Component Modelling and Architectural Pattern Selection for a Self-Service Coffee Kiosk

Introduction

This report aims to discuss the details of completed Lab 3, which involves selecting the most appropriate software architecture for the developed system and displaying the design in the form of a UML component diagram. The main idea of this laboratory is that it is necessary to evaluate the system's decomposition into components, their responsibilities, and interaction before proceeding to code writing. As a result, this document will demonstrate how the self-service coffee kiosk system designed in the previous laboratory can be presented in the form of a component structure, starting from the basic requirements and ending with a specific choice of architecture.

Understanding the Scenario

What the Kiosk Must Do

The kiosk provides a customer with a complete ordering experience. The customer chooses one of three available options of coffee, which are espresso, americano, and latte, and the size of the drink, small or large. The payment for the ordered coffee is accepted only by credit card, and after the payment, a machine prints a receipt, which includes the details of the order.

Technical Constraints

The software must be able to respond to touch screen input, communicate with the receipt printer hardware, and store menu and pricing information so that each order is priced correctly. These requirements dictate the design of the software since each addresses a different aspect over which the system must preside.

Comparing Architectural Styles

Layered Architecture

A layered design is a design where the software is organized into horizontal layers, most commonly a presentation layer, a business layer and a data layer. It has well-defined

responsibilities and is easy to understand, making it ideal for traditional applications. Its shortcomings are some additional overhead as requests have to travel through the different layers and the challenge of scaling individual components.

Microservices Architecture

Microservices can split an application into smaller parts which are easier to maintain and deploy. However, the costs of these services increase with the number of services, and there are problems such as delays in networks and data consistency that must be addressed. It's a wash for a single kiosk.

Client-Server Architecture

In a client-server design the main server serves many clients. The control is centralized and easier to manage, but the server could be a point of failure and a possible bottleneck. This style has in mind the central machine plus many clients, whereas the kiosk is one self-contained device.

Style	Fits the kiosk	Reason
Layered	Yes	Clear flow from interface to logic on data on a single device
Microservices	No	Network delays and deployment effort are too heavy for one kiosk
Client-server	Partly	Assumes a central server with many clients

Selected Architecture and Reasons

Why Layered Architecture Was Chosen

The two reasons why layered architecture was selected for the kiosk include that it clearly isolates the components in accordance with the system's necessities. The touch screen, management of the orders and payments, and storage of the menu are several aspects that can be separated and controlled independently using this approach. Thus, it will become possible to redesign the interface without affecting the payment system and its algorithms. The second reason is that it was found ideal and appropriate for the scale of the project. The kiosk is a

small application with a narrow scope, and it makes sense to use a simple design that can be easily developed, deployed, and maintained on a single computer rather than a distributed system.

Security Advantage

Card data is processed only by the payment component, which is accessed via a separate interface. The user interface does not have access to card data either directly or indirectly, as there is no connection between the interface and the database. All requests are processed by the order manager, which can be reviewed at the first stage. As a result, the attack surface is minimized, and code is reviewed.

Performance Benefit

All components operate in the same device, which ensures the communication between the levels is performed within the device, thus eliminating the network latency. The menu and price information are stored locally to the system which makes its retrieval faster and allows a responsive touch-screen interface even under heavy load conditions.

Components and Interfaces

The Five Components

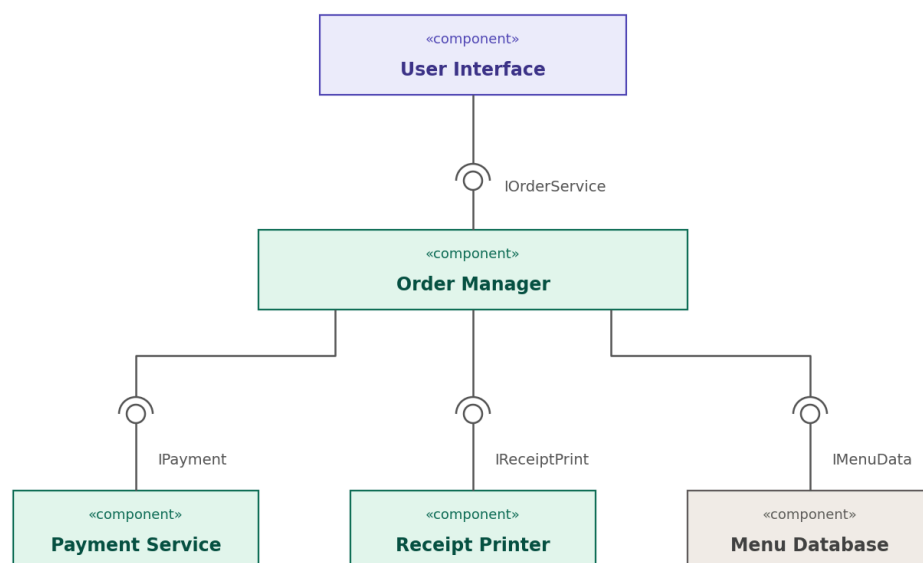
The design involves such components as the touch screen, order manager, payment service, receipt printer, and menu database. The user interface operates the touch screen where the customer chooses his/her desired drinks, sizes, and approves the purchase. In turn, the order manager arranges all the incoming data about the customer's choice, calculates the amount to be paid, decides when to prompt for payment and prints the order. The payment service processes the credit card payment and informs whether it has been approved. Additionally, the receipt printer manages the actual printing process, whereas the menu database stores the information about coffee types, sizes, and prices.

Layer	Components
Presentation	User Interface
Business	Order Manager,Payment Service,Receipt Printer
Data	Menu Database

The Four Interfaces

Four interfaces link these components. The order service interface is offered by the order manager and is used by the user interface and carries the customer's selections and the order total. The payment interface is offered by the payment service and is used by the order manager, and it carries the payment request and the approval result. The receipt print interface is offered by the receipt printer and is used by the order manager, and it carries the order details that need to be printed. The menu data interface is offered by the menu database and is used by the order manager, and it carries the menu and price queries. In every case the component offering a service displays it as a ball, and the component depending on that service displays it as a socket.

Interface	Provided By	Required By	Data Carried
IOrderService	Order Manager	User Interface	Selections and Order totals
IPayment	Payment Service	Order Manager	Payment request and approval result
IReceiptPrint	Receipt Printer	Order Manager	Order details to print
IMenuData	Menu Database	Order Manager	Menu and price queries



Ball = provided interface, socket = required interface

Answers to the Lab Steps

Step 1 Scenario Review

The functional requirements of the kiosk are relatively simple. It should show the list of available coffee options and allow customers to choose the desired coffee, customize the drink size, pay for the ordered item with the card and print the receipt. In addition, it should be possible to find the prices of coffee and other items in the database to ensure that the correct amount was charged to the client's card.

The non-functional requirements define additional standards that should be met by the new system. First, the screen should be intuitive and easy to navigate, even for first-time visitors. The card data should be kept confidential and should not be shared with third parties. The system should be highly available and should not be down during the store's open hours. It should also be easy to update the prices and the menu. Finally, the new kiosk should be compatible with the touch screen and the receipt printer hardware.

The project has several challenges that can be grouped into three main categories. First, in terms of performance, the system should be efficient enough to allow the next customer to make an order while waiting in line. As for the usability, the kiosk should provide a consistent and intuitive ordering experience that would be easy to understand by any customer segment. Finally, in terms of security, the system should ensure that the card details are kept confidential and are not shared with third parties. One of the technical challenges includes the compatibility of the kiosk with the hardware. It should not crash when the receipt printer is out of paper or the card reader malfunctions.

Step 2 Architectural Style Analysis

The layered style is perfect for the kiosk case. Its major benefit is that the screen, the ordering logic and the stored data are separated, which is easy to maintain and test. The only minor drawback, which is extra overhead between levels and inability to scale any layer independently, is almost non-existent because the kiosk is a small database running on one computer.

Microservices allow independent scaling, isolation of faults and using various technologies for each service. However, there is no need for a kiosk with three drinks and two sizes. Network latency, complex operations and data consistency would make the machine slower and harder to maintain than necessary.

The client-server architecture can centralize control and help simplify operations and data maintenance, but it can also make the system dependent on the server and create a single point of failure. This matters for the kiosk because if the server stops working, all clients will be disabled, but the machine needs to be able to take the order in any case.

Step 3 Architecture Selection and Component Identification

Having considered three possible styles, I have chosen to use the layered architecture. There are five components in the application. I have got the first two components, Order Manager and Payment Service, from the handout. The other three components are the User Interface, the Receipt Printer, and the Menu Database. The User Interface is part of the presentation layer. The Order Manager, Payment Service, and Receipt Printer belong to the business layer, while the Menu Database constitutes the data layer.

Step 4 Component Diagram

The component diagram is drawn using UML's standard component notation. There are four interfaces between the components. As per the notations, each interface is represented by a ball on the component providing the interface and a socket on the component that requires the interface. Hence, the "Order service" interface connects the User Interface and the Order Manager, and the "Payment", "Receipt print", and "Menu data" interfaces connect the Order Manager to the Payment Service, Receipt Printer, and Menu Database components. Overall, the component diagram was developed to illustrate that the Order Manager is the only component to which all requests are directed.

Step 5 Written Justification

The Self-Service Coffee Kiosk system is organized as a layered system for two reasons. First, this design helps to separate screen-specific processing, business logic, and storage of data. Second, the system can be considered small enough to reside on a single device. In terms of security, card data is kept isolated in the Payment Service component. In terms of performance, the menu database is also isolated to a single device, which will be able to handle requests faster than any other system that involves networking between multiple devices.

Conclusion

The exercise is illustrative of how the choice of architecture relates to the structure of the system. In particular, for the coffee kiosk, the layered approach creates a good system structure because it provides a clear separation of the functions, protects the payment data and ensures fast response for the display without incurring the overhead of more substantial distributed approaches. The component diagram reflects this viewpoint by explicitly stating the components, their services, dependencies, and data flows.